

Matrix Tree Theorem

1. What is a spanning tree?
In graph theory, a spanning tree of a connected, undirected graph G is a subgraph that is a tree and connects all the vertices of G. A graph can have multiple spanning trees.
2. The Matrix Tree Theorem is a powerful tool in graph theory that provides a way to calculate the number of spanning trees of a given graph.
The Theorem states that for a connected graph G with n vertices, the number of spanning trees is equal to the determinant of any cofactor of its Laplacian matrix.
3. The Laplacian matrix L of a graph G is defined as $L = D - A$, where:
 - D is the degree matrix, a diagonal matrix where $D[i, i]$ is the degree of vertex i.
 - A is the adjacency matrix, where $A[i, j]$ is the number of edges between vertices i and j.
4. A cofactor of a matrix is the determinant of the submatrix obtained by removing one row and one column, multiplied by $(-1)^{i+j}$ where i and j are the row and column indices removed.

The theorem states that the determinant of *any* cofactor of the Laplacian matrix of a connected graph is equal to the number of spanning trees.

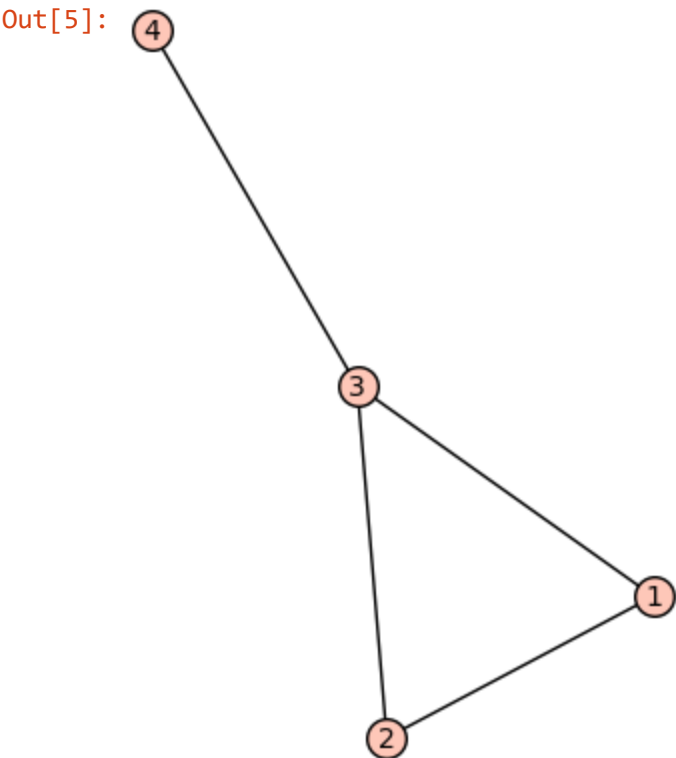
Let us first understand how the graph looks and what are it's different spanning trees
Let us begin with an example of an undirected and connected graph with 4 vertices and edge set given as {(1, 2), (1, 3), (2, 3), (3, 4)}

In [4]:

```
import numpy as np
import ast
```

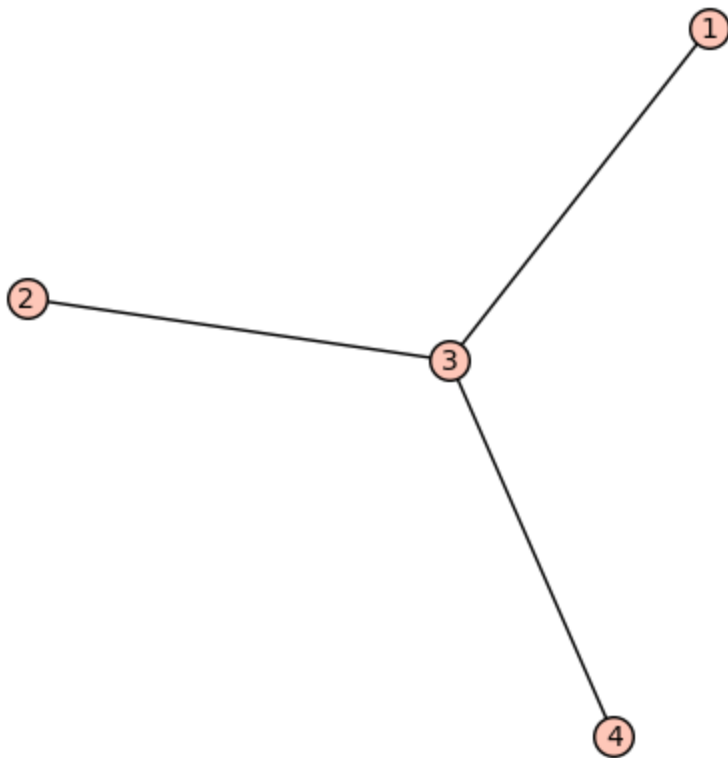
In [5]:

```
num_vertices = 4
edges = [(1, 2), (1, 3), (2, 3), (3, 4)]
G = Graph(edges)
G.show()
print(f"Number of spanning trees of graph G is : {G.spanning_trees_count()}")
spanning_trees = G.spanning_trees()
for (i, tree) in enumerate(spanning_trees, start=1):
    print(f"Spanning tree {i}:")
    tree.show()
```

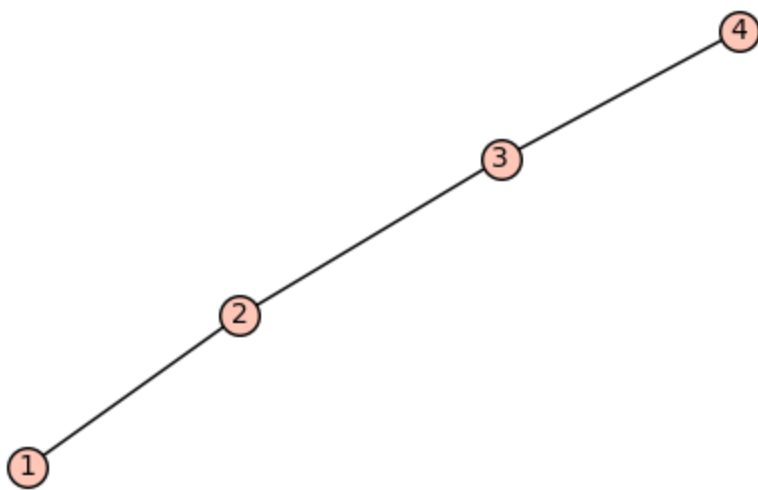


Number of spanning trees of graph G is : 3

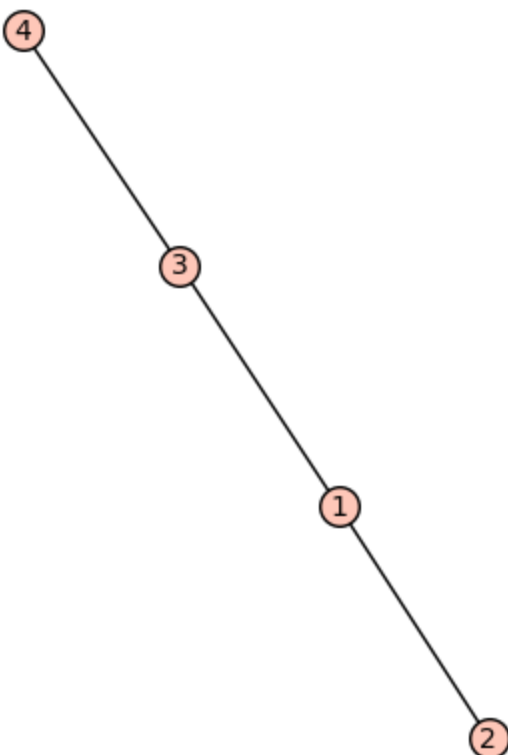
Spanning tree 1:



Spanning tree 2:



Spanning tree 3:



Now let us count the number of spanning trees from the cofactor of the Laplacian matrix
For this we will require Adjacency and Degree matrix

```
In [6]: #Laplacian matrix

def adj_matrix(edges, num_vertices):

    # create an all zero matrix
    matrix = np.zeros((num_vertices, num_vertices), dtype=int)
```

```

#fill the appropriate entries with 1's and 0's
for (u, v) in edges:
    matrix[u-1, v-1] = 1    # since the indexing starts from 0
    matrix[v-1, u-1] = 1

return matrix

```

```

In [7]: edges = [(1, 2), (1, 3), (2, 3), (3, 4)]
        num_vertices = 4
        A= adj_matrix(edges, num_vertices)
        print(A)

```

```

Out[7]: [[0 1 1 0]
         [1 0 1 0]
         [1 1 0 1]
         [0 0 1 0]]

```

```

In [8]: # Next we want the degree matrix

def deg_matrix(adjacency_matrix):

    # calculate the degree of each vertex by summing the values in each row
    degree = np.sum(adjacency_matrix, axis=1)

    # create the degree matrix
    degree_matrix = np.diag(degree)

    return degree_matrix

D= deg_matrix(A)
print(D)

```

```

Out[8]: [[2 0 0 0]
         [0 2 0 0]
         [0 0 3 0]
         [0 0 0 1]]

```

```

In [9]: #now we get the laplacian matrix
        L=D-A
        print(L)

```

```

Out[9]: [[ 2 -1 -1  0]
         [-1  2 -1  0]
         [-1 -1  3 -1]
         [ 0  0 -1  1]]

```

Let us consider the minor matrix M of the Laplacian matrix obtained by removing the first row and first column

```

In [10]: #the determinant of M
        M= L[1:,1:]
        print(M)
        det= int(np.linalg.det(M))
        print("The determinant of M is :",det)

        #check whether the theorem statement holds
        det==G.spanning_trees_count()

```

```

Out[10]: [[ 2 -1  0]
          [-1  3 -1]
          [ 0 -1  1]]
The determinant of M is : 3

True

```

Let us generalize this by taking input graphs from the user in terms of vertices and edges.

```

In [12]: #inputs from the user
        num_vertices_input= input("Enter the number of vertices:")
        num_vertices= int(num_vertices_input)
        edges_input_str = input("Enter the edges (e.g., (1,2),(1,3),(2,3)): ")

```

```

edges =list (ast.literal_eval(edges_input_str)) #the input edges which is string is
taken in list as tuples

#getting the graph
G = Graph(edges)
print("The graph G from given input of vertices and edges is:")
G.show()

# number of spanning trees
print(f"Number of spanning trees of graph G is {G.spanning_trees_count()}.")

#gettng the different spanning trees
spanning_trees = G.spanning_trees()
for (i, tree) in enumerate(spanning_trees, start=1):
    print(f"Spanning tree {i}:")
    tree.show()

#adjacency matrix
A= adj_matrix(edges, num_vertices)
print("The adjacency matrix is \nA =",A)

#degree matrix
D= deg_matrix(A)
print("The degree matrix is \nD =",D)

#laplacian matrix
L=D-A
print("The laplacian matrix is \nL = D-A =",L)

#the minor of the laplacian and its determinant
M= L[1:,1:]
print("The minor matrix afer removing the first column and first row is \nM=",M)

det= round(np.linalg.det(M))
print ("The determinant of this minor=",det)

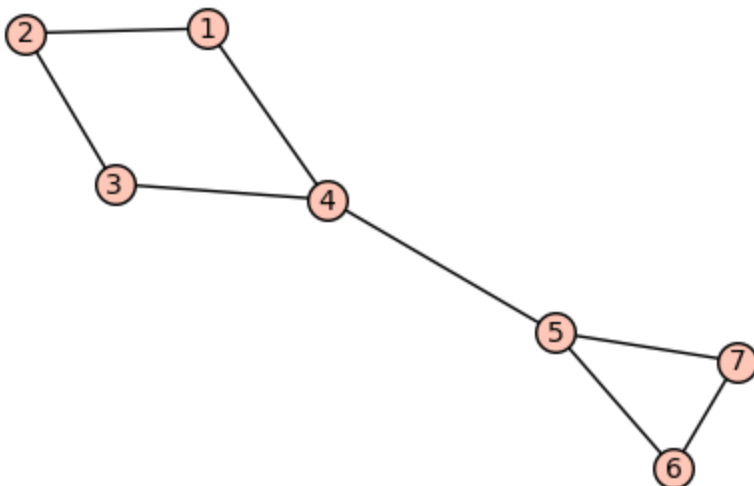
#checking the result
print("Is number of spanning trees of G = determinant of M :",det ==
G.spanning_trees_count())

```

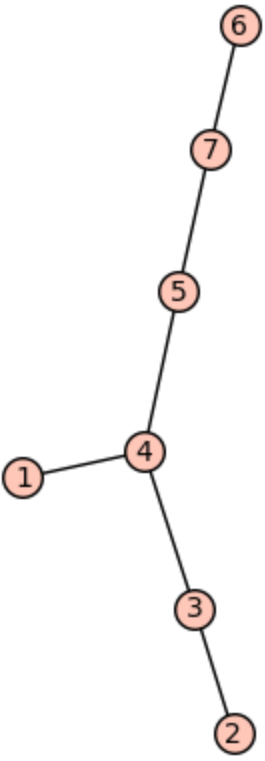
Out[12]: Enter the number of vertices:

Enter the edges (e.g., (1,2),(1,3),(2,3)):

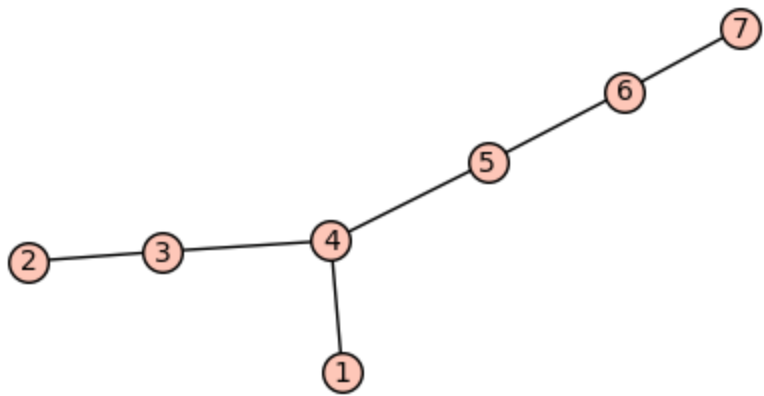
The graph G from given input of vertices and edges is:



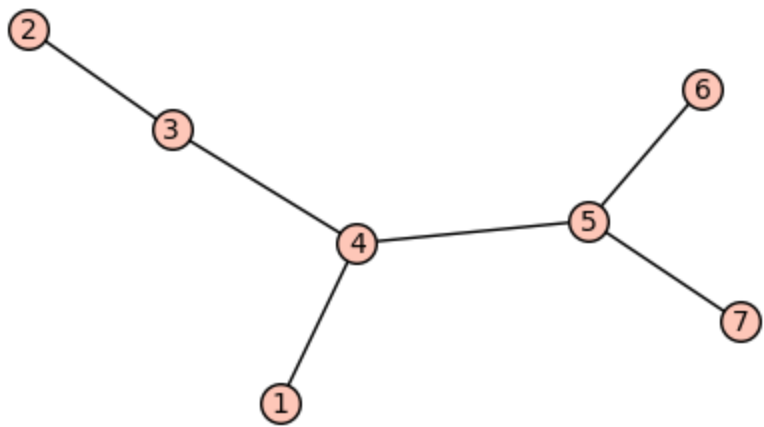
Number of spanning trees of graph G is 12.
Spanning tree 1:



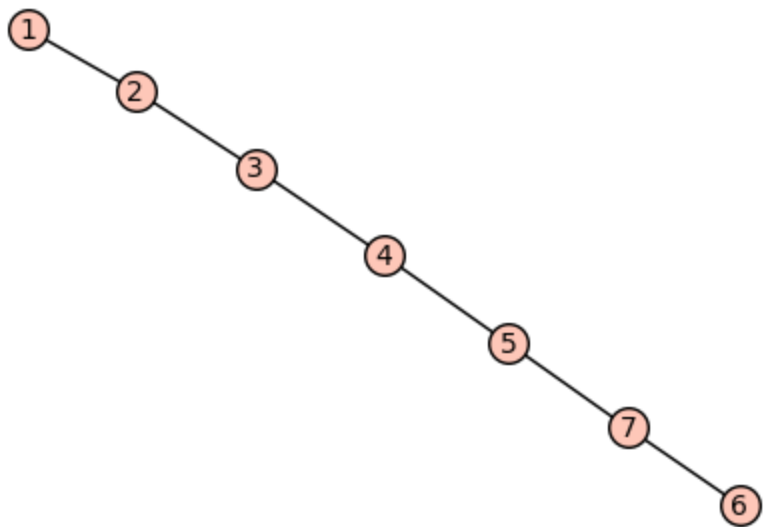
Spanning tree 2:



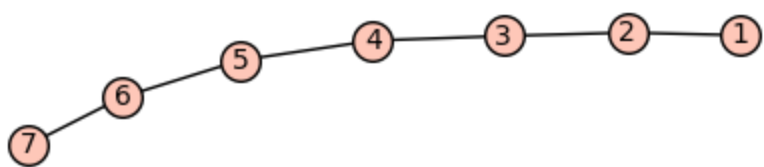
Spanning tree 3:



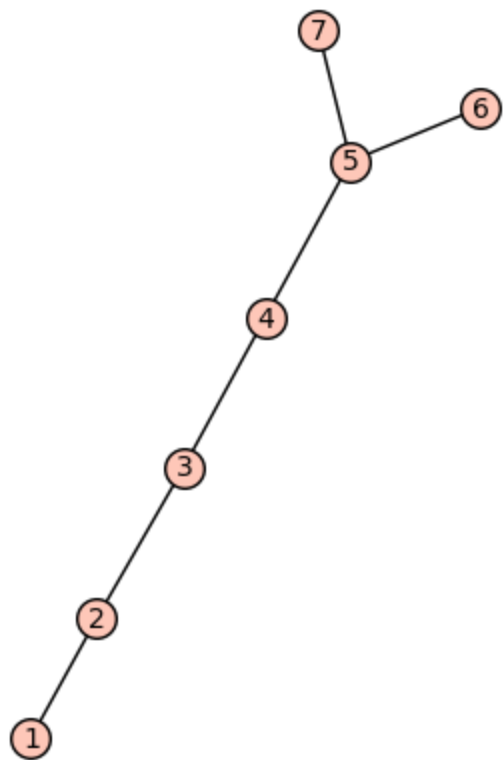
Spanning tree 4:



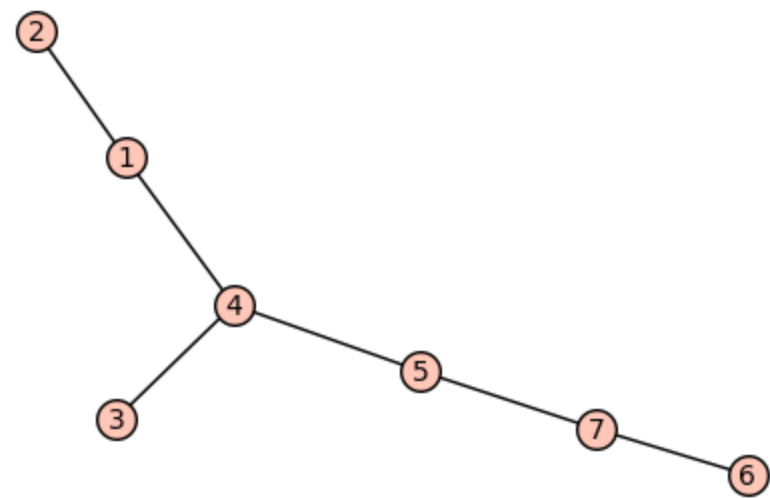
Spanning tree 5:



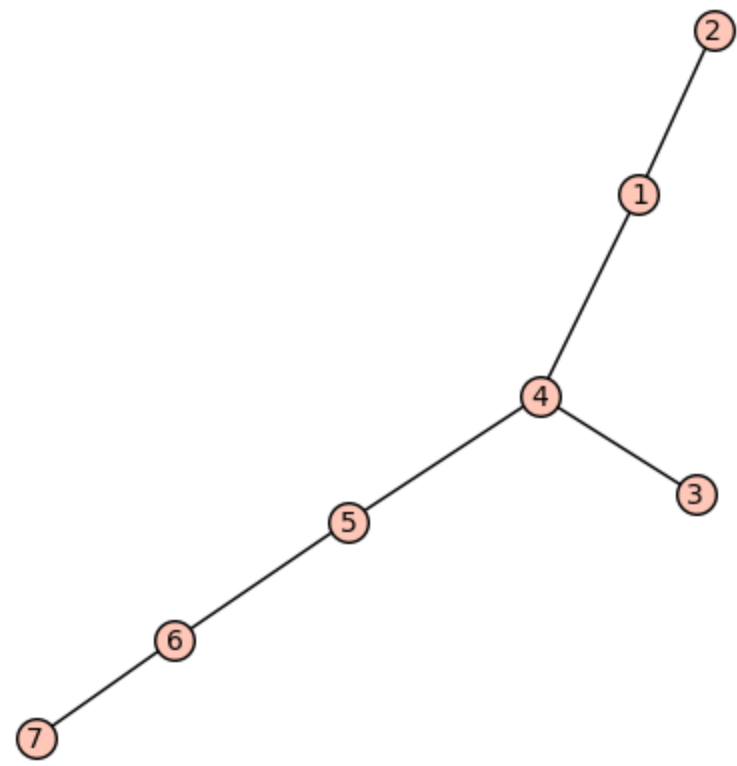
Spanning tree 6:



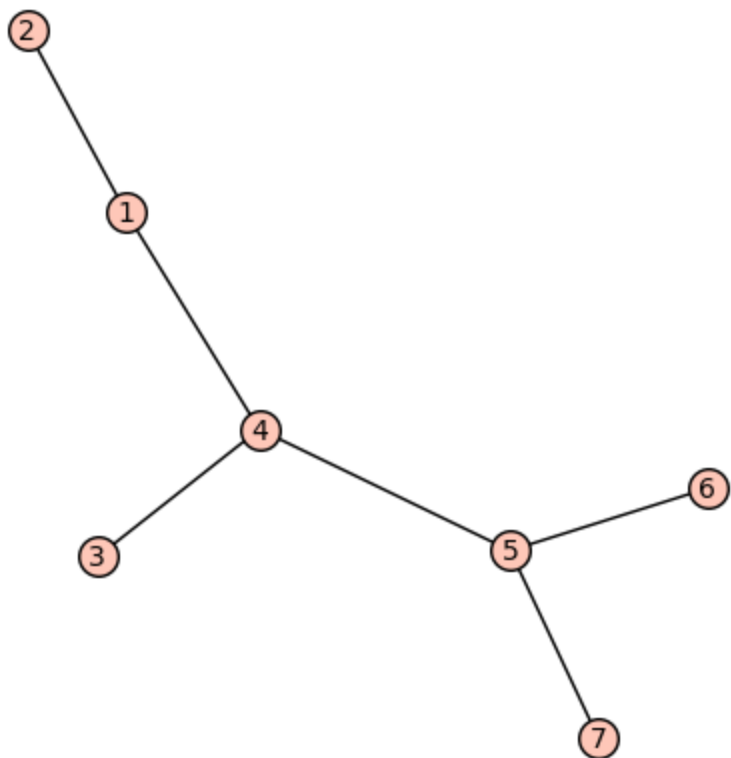
Spanning tree 7:



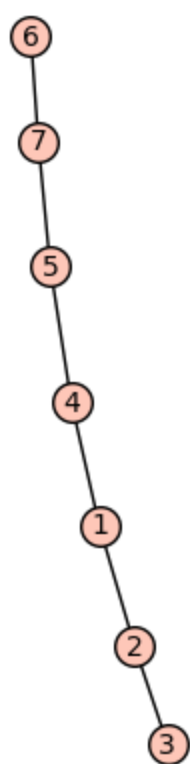
Spanning tree 8:



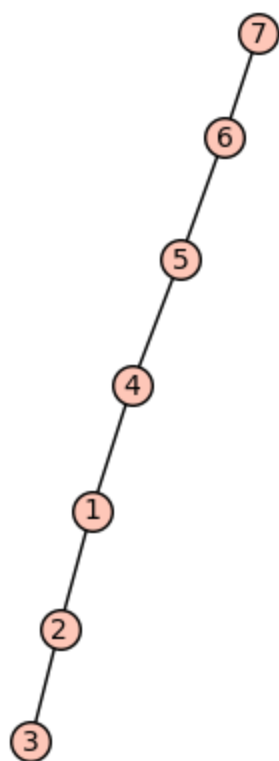
Spanning tree 9:



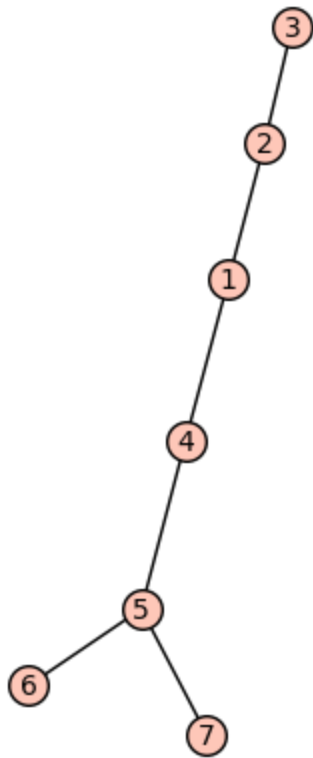
Spanning tree 10:



Spanning tree 11:



Spanning tree 12:



The adjacency matrix is

```

A = [[0 1 0 1 0 0 0]
      [1 0 1 0 0 0 0]
      [0 1 0 1 0 0 0]
      [1 0 1 0 1 0 0]
      [0 0 0 1 0 1 1]
      [0 0 0 0 1 0 1]
      [0 0 0 0 1 1 0]]
  
```

The degree matrix is

```

D = [[2 0 0 0 0 0 0]
      [0 2 0 0 0 0 0]
      [0 0 2 0 0 0 0]
      [0 0 0 3 0 0 0]
      [0 0 0 0 3 0 0]
      [0 0 0 0 0 2 0]
      [0 0 0 0 0 0 2]]
  
```

The laplacian matrix is

```

L = D-A = [[ 2 -1  0 -1  0  0  0]
            [-1  2 -1  0  0  0  0]
            [ 0 -1  2 -1  0  0  0]
            [-1  0 -1  3 -1  0  0]
            [ 0  0  0 -1  3 -1 -1]
            [ 0  0  0  0 -1  2 -1]
            [ 0  0  0  0 -1 -1  2]]
  
```

The minor matrix after removing the first column and first row is

```

M= [[ 2 -1  0  0  0  0]
     [-1  2 -1  0  0  0]
     [ 0 -1  3 -1  0  0]
     [ 0  0 -1  3 -1 -1]
     [ 0  0  0 -1  2 -1]
     [ 0  0  0 -1 -1  2]]
  
```

The determinant of this minor= 12.0

Is number of spanning trees of G = determinant of M : True

Why does this happens?

To understand how this theorem works we need to understand the Cauchy-Binet Formula.

This formula tells us how to compute the determinant of a product of non-square matrices. It states that the determinant is a sum over all possible ways to choose a certain number of columns from the first matrix and the corresponding rows from the second.\

To prove the theorem, one can perform the following steps:

1. Represent Edges as Vectors:

Assign an arbitrary orientation to each edge in the graph. Create a matrix called the incidence matrix B.

For a graph with n vertices and m edges, B is an n×m matrix where B_{ve} describes the relationship between vertex v and edge e:

+1 if v is the head of edge e.

-1 if v is the tail of edge e.

0 otherwise.

2. The Laplacian as a Product:

It can be shown that the Laplacian matrix L is the product of this incidence matrix and its transpose: $L = BB^T$. This is a crucial algebraic step that reformulates the Laplacian in terms of its constituent edges.

3. Applying Cauchy-Binet:

The Matrix Tree Theorem asks for the determinant of a cofactor of L , which means we remove a row and a column (let's say the last ones) to get a submatrix L_0 . If we call the corresponding submatrix of the incidence matrix B_0 (with one row removed), then $L_0 = B_0B_0^T$. Now we can apply the Cauchy-Binet formula to find $\det(L_0)$:

$\det(L_0) = \det(B_0B_0^T) = \sum_S (\det(B_S))^2$. This comes from the Cauchy-Binet formula.

Here, the sum is over all possible ways to choose $n-1$ columns from B_0 . Each choice of $n-1$ columns corresponds to choosing a subgraph with $n-1$ edges. B_S is the square submatrix of B_0 formed by those chosen columns (edges).

4. The Final Step:

What Does $\det(B_S)$ Mean? This is the magic. It turns out that the determinant of this submatrix, $\det(B_S)$, has a specific value:

It is ± 1 if the $n-1$ edges selected form a spanning tree. It is 0 if the $n-1$ edges selected form a cycle (making them linearly dependent).

Since the formula squares this value $\det(B_S)^2$, the result is either 1 (if the edges form a tree) or 0 (if they do not).

Therefore, when we calculate the determinant of the Laplacian cofactor, the Cauchy-Binet formula gives us a combinatorial sum. This sum iterates through all possible combinations of $n-1$ edges from the graph. The properties of the incidence matrix ensure that each term in this sum evaluates to 1 if the chosen set of edges is a spanning tree and 0 otherwise. In the end, the determinant is simply adding up a "1" for every single spanning tree in the graph.

Conclusion:

The Matrix Tree Theorem works because the determinant, obtained through the Cauchy-Binet formula, provides a good algebraic method for selecting all possible subgraphs of the correct size ($n-1$ edges) and filtering out only those that are acyclic (the spanning trees). The Laplacian matrix is the specific construction that encodes the graph's structure in just the right way for this method to function correctly.